

올리브영형 커머스 플랫폼 백엔드 설계서

1. 문서 목적

본 문서는 올리브영과 유사한 **헬스·뷰티 커머스 플랫폼**을 구축하기 위한 백엔드 설계 문서이다.

단순한 쇼핑몰이 아니라, 다음과 같은 특성을 가진 커머스 서비스를 목표로 한다.

- 수많은 상품과 옵션을 안정적으로 관리한다.
- 검색, 카테고리, 랭킹, 추천을 통해 사용자가 빠르게 상품을 찾을 수 있게 한다.
- 장바구니, 주문, 결제, 쿠폰, 포인트, 배송을 하나의 흐름으로 안정적으로 처리한다.
- 재고 부족, 결제 실패, 주문 취소, 환불 같은 예외 상황을 정확하게 처리한다.
- 관리자와 운영자가 상품, 재고, 프로모션, 주문, 배송 상태를 효율적으로 관리할 수 있게 한다.
- 트래픽 증가와 이벤트성 주문 폭주에도 장애 없이 확장 가능한 구조를 가진다.

이 문서는 실제 설계 자료, 면접 발표 자료, 프로젝트 기획서, 개발 착수 문서로 활용할 수 있도록 쉽고 간결하게 작성한다.

2. 기획 의도

2.1 왜 커머스 플랫폼을 설계하는가

커머스 서비스는 사용자의 탐색, 구매, 결제, 배송, 재구매까지 이어지는 복잡한 비즈니스 흐름을 포함한다.

특히 올리브영과 같은 헬스·뷰티 커머스는 다음과 같은 특징이 있다.

- 상품 수가 많고 카테고리가 세분화되어 있다.
- 같은 상품이라도 옵션, 용량, 세트 구성, 행사 여부가 다양하다.
- 쿠폰, 할인, 적립금, 멤버십 혜택이 구매 결정에 큰 영향을 준다.
- 오프라인 매장 재고와 온라인 재고가 함께 고려될 수 있다.
- 인기 상품, 행사 상품, 신상품에 트래픽이 집중될 수 있다.
- 주문 이후 배송, 교환, 반품, 환불 상태 관리가 중요하다.

따라서 커머스 백엔드는 단순 CRUD가 아니라 **정확성, 확장성, 성능, 운영 편의성**을 모두 고려해야 한다.

2.2 설계 목표

본 설계의 목표는 다음과 같다.

1. 사용자는 원하는 상품을 빠르게 찾고 안정적으로 구매할 수 있어야 한다.
2. 주문과 결제는 데이터 정확성이 깨지지 않도록 처리되어야 한다.
3. 재고는 중복 판매가 발생하지 않도록 안전하게 관리되어야 한다.
4. 쿠폰, 할인, 포인트 정책은 유연하게 변경 가능해야 한다.
5. 서비스 트래픽이 증가해도 주요 기능이 독립적으로 확장 가능해야 한다.
6. 운영자는 관리자 시스템에서 상품, 주문, 프로모션을 쉽게 관리할 수 있어야 한다.
7. 장애 발생 시 문제 원인을 빠르게 파악하고 복구할 수 있어야 한다.

3. 서비스 범위

3.1 사용자 서비스

사용자 앱 또는 웹에서 제공하는 주요 기능은 다음과 같다.

- 회원가입 / 로그인
- 상품 목록 조회
- 상품 상세 조회
- 상품 검색
- 카테고리 탐색
- 장바구니 담기
- 바로 구매
- 쿠폰 적용
- 포인트 사용 및 적립
- 주문 생성
- 결제 요청
- 주문 취소
- 배송 조회
- 리뷰 작성
- 찜하기
- 마이페이지

3.2 관리자 서비스

운영자와 관리자가 사용하는 백오피스 기능은 다음과 같다.

- 상품 등록 / 수정 / 판매 상태 변경
- 카테고리 관리
- 브랜드 관리
- 상품 옵션 관리
- 재고 등록 / 수정
- 가격 및 할인 관리
- 쿠폰 발급 / 중지
- 기획전 관리
- 주문 조회
- 주문 상태 변경
- 배송 상태 관리
- 환불 / 반품 처리
- 사용자 문의 관리
- 매출 및 통계 조회

3.3 외부 연동 범위

커머스 플랫폼은 여러 외부 시스템과 연동된다.

- 결제 대행사: 카드, 간편결제, 가상계좌 결제 처리
- 배송사: 운송장 발급, 배송 상태 조회
- 알림 서비스: SMS, 카카오톡, 이메일, 앱 푸시
- 검색 엔진: 상품 검색 품질 개선
- 이미지 저장소: 상품 이미지, 리뷰 이미지 저장

- 인증 서비스: 소셜 로그인, 본인 인증
 - 데이터 분석 도구: 사용자 행동 분석, 매출 분석
-

4. 핵심 요구사항

4.1 기능 요구사항

회원

- 사용자는 이메일, 휴대폰, 소셜 계정으로 가입할 수 있다.
- 사용자는 로그인 후 주문, 리뷰, 쿠폰, 포인트 기능을 사용할 수 있다.
- 회원 등급에 따라 할인율, 적립률, 쿠폰 혜택이 달라질 수 있다.

상품

- 상품은 브랜드, 카테고리, 가격, 옵션, 이미지, 설명, 판매 상태를 가진다.
- 상품은 여러 옵션을 가질 수 있다.
- 예: 색상, 용량, 세트 구성
- 상품은 판매중, 품절, 판매중지, 숨김 상태를 가질 수 있다.

검색

- 사용자는 키워드로 상품을 검색할 수 있다.
- 검색 결과는 정확도, 판매량, 인기순, 최신순, 낮은 가격순, 높은 가격순으로 정렬할 수 있다.
- 오타, 띄어쓰기, 초성 검색까지 확장 가능하도록 설계한다.

장바구니

- 사용자는 상품 옵션과 수량을 장바구니에 담을 수 있다.
- 장바구니에는 현재 가격, 할인 가능 여부, 재고 상태가 표시되어야 한다.
- 주문 시점에는 장바구니 가격이 아니라 최신 가격과 재고를 다시 검증해야 한다.

주문

- 사용자는 여러 상품을 한 번에 주문할 수 있다.
- 주문은 주문 생성, 결제 대기, 결제 완료, 상품 준비, 배송중, 배송 완료, 취소, 환불 상태를 가진다.
- 주문 생성 시 가격, 할인, 쿠폰, 포인트, 배송비가 확정되어야 한다.

결제

- 결제는 외부 PG사와 연동한다.
- 결제 성공 시 주문 상태를 결제 완료로 변경한다.
- 결제 실패 시 주문은 결제 실패 또는 결제 대기 상태로 남긴다.
- 결제 결과는 PG사의 콜백을 기준으로 최종 확정한다.

재고

- 주문 시 재고를 선점하거나 차감해야 한다.
- 결제 실패 또는 주문 취소 시 재고를 복구해야 한다.
- 동시 주문 상황에서도 재고가 음수가 되지 않도록 처리해야 한다.

쿠폰 / 포인트

- 쿠폰은 발급 대상, 사용 조건, 할인 금액, 유효기간을 가진다.
- 포인트는 적립, 사용, 취소, 만료 이력을 관리해야 한다.
- 쿠폰과 포인트는 주문 금액 계산 시 정확히 반영되어야 한다.

배송

- 결제 완료 후 배송 준비 상태로 전환된다.
- 운송장 등록 후 배송중 상태가 된다.
- 배송사 연동을 통해 배송 완료 여부를 업데이트한다.

리뷰

- 구매 완료한 사용자만 리뷰를 작성할 수 있다.
- 리뷰는 별점, 텍스트, 이미지 정보를 가진다.
- 리뷰는 상품 상세의 평점 계산에 활용된다.

4.2 비기능 요구사항

성능

- 상품 목록과 상세 조회는 빠르게 응답해야 한다.
- 인기 상품, 카테고리 목록, 메인 페이지 데이터는 캐싱한다.
- 검색은 전문 검색 엔진을 사용하여 빠른 응답을 보장한다.

확장성

- 회원, 상품, 주문, 결제, 재고, 프로모션 도메인은 독립적으로 확장 가능해야 한다.
- 초기에는 모듈형 모놀리식으로 시작하고, 트래픽 증가 시 MSA로 분리할 수 있게 설계한다.

안정성

- 결제와 주문은 중복 요청이 발생해도 한 번만 처리되어야 한다.
- 외부 PG사 콜백이 여러 번 들어와도 주문 상태가 잘못 변경되면 안 된다.
- 재고 차감은 동시성 제어가 반드시 필요하다.

보안

- 사용자 비밀번호는 단방향 해시로 저장한다.
- 결제 정보는 직접 저장하지 않고 PG사의 토큰 또는 거래 ID만 저장한다.
- 관리자 API는 권한 기반 접근 제어를 적용한다.
- 개인정보는 암호화 또는 마스킹 처리한다.

운영성

- 모든 주요 요청은 로그와 추적 ID를 남긴다.
- 주문, 결제, 재고 변경 이력은 감사 로그로 관리한다.
- 장애 발생 시 알림이 발송되어야 한다.

5. 전체 아키텍처

5.1 권장 구조

초기 구축 단계에서는 **모듈형 모놀리식 구조**를 권장한다.

이유는 다음과 같다.

- 초기 개발 속도가 빠르다.
- 트랜잭션 관리가 단순하다.
- 도메인 간 호출 복잡도가 낮다.
- 운영 비용이 낮다.
- 나중에 도메인별 서비스 분리가 가능하다.

단, 내부 패키지 구조는 MSA처럼 도메인 단위로 분리한다.

예시 구조:

```
commerce-backend
├─ member
├─ product
├─ search
├─ cart
├─ order
├─ payment
├─ inventory
├─ promotion
├─ delivery
├─ review
├─ admin
└─ common
```

5.2 논리 아키텍처

```
[Client]
├─ Mobile App
├─ Web
└─ Admin Web

↓

[API Gateway / Load Balancer]

↓

[Backend Application]
```

- └─ Member Module
- └─ Product Module
- └─ Search Module
- └─ Cart Module
- └─ Order Module
- └─ Payment Module
- └─ Inventory Module
- └─ Promotion Module
- └─ Delivery Module
- └─ Review Module

↓

[Storage]

- └─ RDBMS: 회원, 상품, 주문, 결제, 재고 원장
- └─ Redis: 캐시, 세션, 임시 재고 선점
- └─ Elasticsearch/OpenSearch: 상품 검색
- └─ Object Storage: 이미지 파일
- └─ Message Queue: 비동기 이벤트 처리

↓

[External Systems]

- └─ PG사
- └─ 배송사
- └─ 알림 서비스
- └─ 데이터 분석 도구

6. 도메인 설계

6.1 회원 도메인

역할

회원 도메인은 사용자 계정, 인증, 권한, 배송지, 멤버십 정보를 관리한다.

주요 테이블

- members
- member_addresses
- member_grades
- member_login_histories

핵심 설계 포인트

- 회원 ID는 모든 주문, 리뷰, 쿠폰의 기준이 된다.
- 개인정보는 별도 테이블로 분리하여 보안 수준을 높인다.

- 관리자와 일반 사용자의 권한 체계는 분리한다.

6.2 상품 도메인

역할

상품 도메인은 판매 상품, 브랜드, 카테고리, 옵션, 이미지, 판매 상태를 관리한다.

주요 테이블

- products
- product_options
- product_images
- brands
- categories
- product_category_mapping

상품 상태

DRAFT	: 임시 저장
ON_SALE	: 판매 중
SOLD_OUT	: 품절
STOPPED	: 판매 중지
HIDDEN	: 사용자 화면에서 숨김

핵심 설계 포인트

- 상품과 옵션은 분리한다.
- 재고는 상품이 아니라 옵션 단위로 관리한다.
- 상품 가격은 변경될 수 있으므로 주문 시점의 가격을 주문 상세에 복사 저장한다.
- 상품 이미지는 외부 Object Storage에 저장하고 URL만 DB에 저장한다.

6.3 검색 도메인

역할

검색 도메인은 사용자의 키워드 검색, 필터, 정렬, 자동완성, 인기 검색어를 처리한다.

검색 대상 정보

- 상품명
- 브랜드명
- 카테고리명
- 상품 태그
- 주요 성분
- 상품 설명 요약
- 판매 상태

- 가격
- 평점
- 리뷰 수
- 판매량

핵심 설계 포인트

- DB LIKE 검색은 초기 단계에서는 가능하지만 상품 수가 증가하면 한계가 있다.
- Elasticsearch 또는 OpenSearch를 사용하여 검색 품질과 속도를 개선한다.
- 상품 데이터 변경 시 검색 인덱스도 함께 갱신되어야 한다.
- 검색 인덱스 갱신은 비동기 이벤트 방식으로 처리한다.

6.4 장바구니 도메인

역할

장바구니 도메인은 사용자가 구매 전 담아둔 상품 옵션과 수량을 관리한다.

주요 테이블

- carts
- cart_items

핵심 설계 포인트

- 장바구니는 구매 의사만 표현한다.
- 장바구니에 담긴 가격과 재고는 주문 시점에 다시 검증한다.
- 비회원 장바구니는 세션 또는 Redis 기반으로 관리할 수 있다.
- 회원 로그인 시 비회원 장바구니와 회원 장바구니를 병합할 수 있다.

6.5 주문 도메인

역할

주문 도메인은 사용자의 구매 확정 정보를 관리한다.

주요 테이블

- orders
- order_items
- order_price_summaries
- order_status_histories

주문 상태

CREATED	: 주문 생성
PAYMENT_PENDING	: 결제 대기
PAID	: 결제 완료

PREPARING	: 상품 준비중
SHIPPING	: 배송중
DELIVERED	: 배송 완료
CANCELED	: 주문 취소
REFUND_REQUESTED	: 환불 요청
REFUNDED	: 환불 완료
FAILED	: 주문 실패

핵심 설계 포인트

- 주문 생성 시 주문번호를 발급한다.
- 주문 상세에는 상품명, 옵션명, 가격, 수량을 복사 저장한다.
- 이는 나중에 상품명이 변경되어도 주문 당시 정보를 보존하기 위함이다.
- 주문 상태 변경 이력은 반드시 별도 테이블로 남긴다.
- 결제 완료 전 주문과 결제 완료 후 주문을 명확히 구분한다.

6.6 결제 도메인

역할

결제 도메인은 PG사와 연동하여 결제 요청, 승인, 실패, 취소, 환불을 처리한다.

주요 테이블

- payments
- payment_transactions
- refunds

결제 상태

READY	: 결제 준비
REQUESTED	: 결제 요청
APPROVED	: 결제 승인
FAILED	: 결제 실패
CANCELED	: 결제 취소
REFUNDED	: 환불 완료

핵심 설계 포인트

- 결제 요청에는 고유한 idempotency key를 사용한다.
- PG사 콜백은 여러 번 들어올 수 있으므로 중복 처리 방지가 필요하다.
- 결제 성공 여부는 클라이언트 응답보다 PG사 서버 콜백 또는 검증 API를 기준으로 확정한다.
- 결제 금액은 주문 금액과 반드시 일치해야 한다.

6.7 재고 도메인

역할

재고 도메인은 상품 옵션별 판매 가능 수량을 관리한다.

주요 테이블

- inventories
- inventory_histories
- inventory_reservations

재고 처리 방식

주문 과정에서 재고를 처리하는 방식은 크게 두 가지다.

방식 1. 주문 생성 시 재고 차감

- 주문 생성 시 바로 재고를 차감한다.
- 결제 실패 시 재고를 복구한다.
- 구현이 단순하지만 결제 이탈이 많으면 재고가 자주 묶인다.

방식 2. 주문 생성 시 재고 선점

- 주문 생성 시 일정 시간 동안 재고를 선점한다.
- 결제 완료 시 최종 차감한다.
- 결제 시간이 초과되면 선점 재고를 해제한다.
- 대형 커머스에서는 이 방식이 더 안전하다.

본 설계에서는 **재고 선점 방식**을 권장한다.

핵심 설계 포인트

- 재고는 상품 옵션 단위로 관리한다.
- 동시 주문을 막기 위해 DB row lock 또는 Redis 분산락을 사용한다.
- 재고 변경은 모두 이력으로 남긴다.
- 재고 선점에는 만료 시간이 필요하다.

6.8 프로모션 도메인

역할

프로모션 도메인은 쿠폰, 할인, 기획전, 포인트 정책을 관리한다.

주요 테이블

- coupons
- member_coupons
- promotions
- promotion_products

- points
- point_histories

할인 유형

FIXED_AMOUNT	: 정액 할인
PERCENTAGE	: 정률 할인
FREE_SHIPPING	: 무료 배송
BUY_ONE_GET_ONE	: N+1 행사
MEMBER_GRADE	: 회원 등급 할인

핵심 설계 포인트

- 할인 정책은 자주 변경되므로 주문 로직과 강하게 결합하지 않는다.
- 쿠폰 사용 가능 여부는 주문 생성 시 검증한다.
- 쿠폰은 한 번만 사용 가능하도록 상태 관리가 필요하다.
- 포인트는 잔액만 저장하지 않고 적립, 사용, 취소 이력을 모두 저장한다.

6.9 배송 도메인

역할

배송 도메인은 배송지, 배송비, 운송장, 배송 상태를 관리한다.

주요 테이블

- deliveries
- delivery_addresses
- delivery_status_histories

배송 상태

READY	: 배송 준비
INVOICE	: 운송장 등록
SHIPPING	: 배송중
DELIVERED	: 배송 완료
RETURNING	: 반품중
RETURNED	: 반품 완료

핵심 설계 포인트

- 주문과 배송은 1:1 또는 1:N 관계가 될 수 있다.
- 여러 상품이 서로 다른 물류센터에서 출고되면 배송이 분리될 수 있다.
- 배송 상태 변경 이력은 별도로 저장한다.
- 배송사 API 장애 시 재시도 처리가 필요하다.

6.10 리뷰 도메인

역할

리뷰 도메인은 구매자의 상품 평가를 관리한다.

주요 테이블

- reviews
- review_images
- review_reports

핵심 설계 포인트

- 구매 완료한 주문 상품에 대해서만 리뷰 작성이 가능해야 한다.
- 리뷰 작성 가능 여부는 order_item 기준으로 판단한다.
- 리뷰 평점은 상품 상세 조회 성능을 위해 집계 테이블 또는 캐시에 저장할 수 있다.
- 신고된 리뷰는 관리자 검수 후 숨김 처리할 수 있다.

7. 데이터베이스 설계 초안

7.1 회원 테이블

```
CREATE TABLE members (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255),  
  name VARCHAR(100) NOT NULL,  
  phone VARCHAR(30),  
  status VARCHAR(30) NOT NULL,  
  grade_id BIGINT,  
  created_at DATETIME NOT NULL,  
  updated_at DATETIME NOT NULL  
);
```

7.2 상품 테이블

```
CREATE TABLE products (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  brand_id BIGINT,  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  status VARCHAR(30) NOT NULL,  
  base_price DECIMAL(12,2) NOT NULL,  
  sale_price DECIMAL(12,2),
```

```
created_at DATETIME NOT NULL,  
updated_at DATETIME NOT NULL  
);
```

7.3 상품 옵션 테이블

```
CREATE TABLE product_options (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  product_id BIGINT NOT NULL,  
  option_name VARCHAR(255) NOT NULL,  
  option_price DECIMAL(12,2) NOT NULL,  
  status VARCHAR(30) NOT NULL,  
  created_at DATETIME NOT NULL,  
  updated_at DATETIME NOT NULL  
);
```

7.4 재고 테이블

```
CREATE TABLE inventories (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  product_option_id BIGINT NOT NULL UNIQUE,  
  total_quantity INT NOT NULL,  
  reserved_quantity INT NOT NULL DEFAULT 0,  
  available_quantity INT NOT NULL,  
  updated_at DATETIME NOT NULL  
);
```

7.5 주문 테이블

```
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  order_no VARCHAR(50) UNIQUE NOT NULL,  
  member_id BIGINT NOT NULL,  
  status VARCHAR(30) NOT NULL,  
  total_product_amount DECIMAL(12,2) NOT NULL,  
  discount_amount DECIMAL(12,2) NOT NULL,  
  point_used_amount DECIMAL(12,2) NOT NULL,  
  delivery_fee DECIMAL(12,2) NOT NULL,  
  final_payment_amount DECIMAL(12,2) NOT NULL,  
  created_at DATETIME NOT NULL,  
  updated_at DATETIME NOT NULL  
);
```

7.6 주문 상세 테이블

```
CREATE TABLE order_items (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  order_id BIGINT NOT NULL,  
  product_id BIGINT NOT NULL,  
  product_option_id BIGINT NOT NULL,  
  product_name VARCHAR(255) NOT NULL,  
  option_name VARCHAR(255) NOT NULL,  
  unit_price DECIMAL(12,2) NOT NULL,  
  quantity INT NOT NULL,  
  total_amount DECIMAL(12,2) NOT NULL,  
  created_at DATETIME NOT NULL  
);
```

7.7 결제 테이블

```
CREATE TABLE payments (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  order_id BIGINT NOT NULL,  
  payment_key VARCHAR(255),  
  pg_provider VARCHAR(50) NOT NULL,  
  method VARCHAR(50) NOT NULL,  
  status VARCHAR(30) NOT NULL,  
  requested_amount DECIMAL(12,2) NOT NULL,  
  approved_amount DECIMAL(12,2),  
  approved_at DATETIME,  
  created_at DATETIME NOT NULL,  
  updated_at DATETIME NOT NULL  
);
```

7.8 쿠폰 테이블

```
CREATE TABLE coupons (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL,  
  discount_type VARCHAR(30) NOT NULL,  
  discount_value DECIMAL(12,2) NOT NULL,  
  min_order_amount DECIMAL(12,2),  
  started_at DATETIME NOT NULL,  
  ended_at DATETIME NOT NULL,  
  status VARCHAR(30) NOT NULL,  
  created_at DATETIME NOT NULL,
```

```
updated_at DATETIME NOT NULL
);
```

8. 주요 API 설계

8.1 상품 API

상품 목록 조회

```
GET /api/products?categoryId=1&sort=popular&page=0&size=20
```

응답 예시:

```
{
  "items": [
    {
      "productId": 1,
      "brandName": "브랜드명",
      "productName": "상품명",
      "salePrice": 18000,
      "originalPrice": 22000,
      "discountRate": 18,
      "thumbnailUrl": "https://...",
      "rating": 4.7,
      "reviewCount": 132
    }
  ],
  "page": 0,
  "size": 20,
  "totalCount": 1000
}
```

상품 상세 조회

```
GET /api/products/{productId}
```

상품 검색

```
GET /api/search/products?keyword=선크림&page=0&size=20
```

8.2 장바구니 API

장바구니 담기

`POST /api/cart/items`

요청 예시:

```
{
  "productOptionId": 10,
  "quantity": 2
}
```

장바구니 조회

`GET /api/cart`

장바구니 수량 변경

`PATCH /api/cart/items/{cartItemId}`

8.3 주문 API

주문 생성

`POST /api/orders`

요청 예시:

```
{
  "items": [
    {
      "productOptionId": 10,
      "quantity": 2
    }
  ],
  "couponId": 5,
  "usePointAmount": 1000,
  "deliveryAddressId": 3
}
```


처리 순서:

1. 회원 검증
2. 상품 판매 상태 검증
3. 재고 검증 및 선점
4. 쿠폰 사용 가능 여부 검증
5. 포인트 사용 가능 여부 검증
6. 최종 결제 금액 계산
7. 주문 생성
8. 결제 요청 정보 생성

8.4 결제 API

결제 승인 요청

`POST /api/payments/confirm`

요청 예시:

```
{
  "orderNo": "ORD202605100001",
  "paymentKey": "pg_payment_key",
  "amount": 35000
}
```

처리 순서:

1. 주문 조회
2. 결제 금액 검증
3. PG사 결제 승인 API 호출
4. 결제 성공 시 주문 상태 PAID 변경
5. 재고 선점 확정
6. 쿠폰 사용 처리
7. 포인트 사용 처리
8. 주문 완료 알림 발송

8.5 관리자 API

상품 등록

`POST /api/admin/products`

상품 수정

```
PATCH /api/admin/products/{productId}
```

주문 상태 변경

```
PATCH /api/admin/orders/{orderId}/status
```

쿠폰 생성

```
POST /api/admin/coupons
```

9. 핵심 비즈니스 플로우

9.1 상품 구매 플로우

상품 조회

- 장바구니 담기
- 주문서 진입
- 상품/가격/재고 재검증
- 쿠폰/포인트 적용
- 주문 생성
- 재고 선점
- 결제 요청
- PG 결제 승인
- 결제 완료
- 재고 확정 차감
- 배송 준비
- 배송중
- 배송 완료

9.2 주문 취소 플로우

주문 조회

- 취소 가능 상태 확인
- PG 결제 취소 요청
- 결제 취소 성공
- 주문 상태 CANCELED 변경
- 재고 복구
- 쿠폰 복구

- 포인트 복구
- 취소 완료 알림 발송

9.3 결제 실패 플로우

결제 요청

- PG 결제 실패
- 주문 상태 PAYMENT_PENDING 또는 FAILED 변경
- 재고 선점 해제
- 쿠폰 미사용 상태 유지
- 사용자에게 실패 사유 안내

10. 동시성 제어 설계

10.1 왜 동시성 제어가 필요한가

인기 상품이나 특가 상품은 짧은 시간에 많은 사용자가 동시에 주문할 수 있다.

예를 들어 재고가 1개 남은 상품에 대해 10명이 동시에 결제하면, 동시성 제어가 없을 경우 10명 모두 주문 성공 처리될 수 있다.

이 문제를 막기 위해 재고 차감 또는 선점 시점에 동시성 제어가 필요하다.

10.2 권장 방식

초기 방식: DB row lock

```
SELECT * FROM inventories
WHERE product_option_id = ?
FOR UPDATE;
```

장점:

- 구현이 단순하다.
- 트랜잭션 안에서 안전하게 처리할 수 있다.

단점:

- 트래픽이 많아지면 DB 부하가 증가한다.

확장 방식: Redis 분산락

- 재고 옵션 ID를 기준으로 Redis lock을 획득한다.
- 락 획득 후 재고를 확인하고 선점한다.
- 처리가 끝나면 락을 해제한다.

장점:

- DB 락 부하를 줄일 수 있다.
- 서버가 여러 대여도 제어 가능하다.

단점:

- 락 만료, 장애 상황을 세심하게 처리해야 한다.

11. 캐시 설계

11.1 캐시 대상

- 메인 페이지 상품 목록
- 인기 상품 목록
- 카테고리 목록
- 브랜드 목록
- 상품 상세 일부 정보
- 검색어 자동완성
- 인기 검색어
- 회원 등급 정책

11.2 캐시 전략

Cache Aside 전략

1. 애플리케이션이 Redis에서 데이터 조회
2. 캐시에 없으면 DB 조회
3. DB 조회 결과를 Redis에 저장
4. 다음 요청부터 Redis에서 응답

캐시 무효화

상품 가격, 판매 상태, 재고 상태가 변경되면 관련 캐시를 삭제하거나 갱신한다.

주의할 점은 재고처럼 실시간성이 중요한 값은 캐시에만 의존하면 안 된다는 것이다.

12. 이벤트 기반 처리 설계

12.1 왜 이벤트가 필요한가

주문 완료 후에는 여러 후속 작업이 필요하다.

- 주문 완료 알림 발송
- 배송 준비 요청
- 포인트 적립 예정 처리

- 판매량 집계
- 검색 인덱스 갱신
- 추천 데이터 갱신

이 작업들을 주문 API 안에서 모두 동기 처리하면 응답이 느려지고 장애 영향 범위가 커진다.

따라서 주문 완료 후 필요한 부가 작업은 이벤트로 분리한다.

12.2 주요 이벤트

```
OrderCreatedEvent
PaymentApprovedEvent
OrderCanceledEvent
InventoryReservedEvent
InventoryReleasedEvent
ProductUpdatedEvent
DeliveryStartedEvent
DeliveryCompletedEvent
ReviewCreatedEvent
```

12.3 이벤트 처리 예시

```
PaymentApprovedEvent 발생
→ 알림 서비스가 주문 완료 메시지 발송
→ 배송 서비스가 배송 준비 데이터 생성
→ 포인트 서비스가 적립 예정 포인트 생성
→ 통계 서비스가 매출 집계 데이터 업데이트
```

13. 검색 설계

13.1 초기 검색

초기에는 DB 기반 검색으로 시작할 수 있다.

```
SELECT * FROM products
WHERE name LIKE '%선크림%'
AND status = 'ON_SALE';
```

하지만 상품 수가 증가하면 성능과 검색 품질에 한계가 생긴다.

13.2 확장 검색

상품 수가 많아지면 Elasticsearch 또는 OpenSearch를 사용한다.

검색 인덱스에는 다음 정보를 저장한다.

```
{
  "productId": 1,
  "productName": "수분 진정 선크림",
  "brandName": "브랜드명",
  "categoryNames": ["스킨케어", "선크어"],
  "tags": ["선크림", "수분", "민감성"],
  "salePrice": 18000,
  "rating": 4.7,
  "reviewCount": 132,
  "salesCount": 5000,
  "status": "ON_SALE"
}
```

13.3 검색 개선 방향

- 형태소 분석
- 동의어 사전
- 오타 보정
- 인기 검색어 반영
- 개인화 추천 반영
- 품질 상품 후순위 처리
- 리뷰 평점과 판매량 기반 랭킹

14. 보안 설계

14.1 인증

- JWT 또는 세션 기반 인증을 사용한다.
- Access Token과 Refresh Token을 분리한다.
- 관리자 계정은 추가 인증 수단을 적용할 수 있다.

14.2 인가

권한은 역할 기반으로 관리한다.

USER	: 일반 사용자
CS_MANAGER	: 고객센터 담당자
PRODUCT_ADMIN	: 상품 관리자
ORDER_ADMIN	: 주문 관리자
SUPER_ADMIN	: 최고 관리자

14.3 개인정보 보호

- 비밀번호는 bcrypt, Argon2 등 안전한 해시 알고리즘으로 저장한다.
- 전화번호, 주소 등 개인정보는 암호화 또는 마스킹한다.
- 관리자 화면에서는 필요한 정보만 노출한다.
- 개인정보 접근 이력을 남긴다.

14.4 결제 보안

- 카드번호 등 민감한 결제 정보는 직접 저장하지 않는다.
 - PG사에서 제공하는 결제 키, 거래 ID만 저장한다.
 - 결제 승인 전 주문 금액과 결제 금액을 반드시 비교한다.
-

15. 장애 대응 설계

15.1 PG사 장애

상황:

- 결제 요청이 실패하거나 응답이 지연된다.

대응:

- 주문은 결제 대기 상태로 유지한다.
- 사용자가 재시도할 수 있게 한다.
- 일정 시간 후 결제 상태 검증 배치를 수행한다.
- 결제 실패가 확정되면 재고 선점을 해제한다.

15.2 배송사 API 장애

상황:

- 운송장 발급 또는 배송 상태 조회 실패

대응:

- 실패 요청은 재시도 큐에 저장한다.
- 관리자 화면에서 수동 재처리할 수 있게 한다.
- 배송 상태는 마지막 정상 상태를 유지한다.

15.3 검색 엔진 장애

상황:

- 검색 엔진 응답 실패

대응:

- 검색 기능은 장애 안내 메시지를 노출한다.

- 메인 상품 목록은 DB 또는 캐시 기반으로 계속 제공한다.
- 검색 인덱스 갱신 실패 이벤트는 재처리 큐로 보낸다.

15.4 Redis 장애

상황:

- 캐시 또는 분산락 사용 불가

대응:

- 캐시 데이터는 DB 조회로 대체한다.
 - 재고 락은 DB row lock 방식으로 fallback한다.
 - 세션 기반 구조라면 인증 영향도를 별도 점검한다.
-

16. 모니터링 설계

16.1 주요 지표

- API 응답 시간
- API 오류율
- 주문 생성 수
- 결제 성공률
- 결제 실패율
- 재고 선점 실패 수
- 주문 취소율
- 검색 응답 시간
- 검색 결과 없음 비율
- Redis hit ratio
- DB connection 사용률
- 메시지 큐 적체량

16.2 주요 로그

- 로그인 실패 로그
- 상품 가격 변경 로그
- 주문 상태 변경 로그
- 결제 요청 / 승인 / 실패 로그
- 재고 변경 로그
- 쿠폰 사용 로그
- 관리자 작업 로그

16.3 알림 기준 예시

- 결제 실패율이 5분 동안 10% 이상 증가
- 주문 API 오류율이 5분 동안 3% 이상 발생
- DB CPU 사용률 80% 이상 지속
- 메시지 큐 적체량 기준치 초과
- 검색 API 응답 시간 2초 이상 지속

17. 배치 설계

17.1 필요한 배치 작업

- 결제 대기 주문 만료 처리
- 재고 선점 만료 해제
- 쿠폰 만료 처리
- 포인트 만료 처리
- 배송 상태 동기화
- 일 매출 집계
- 상품 랭킹 집계
- 리뷰 평점 집계
- 검색 인덱스 재색인

17.2 배치 예시

매 5분 : 결제 대기 주문 만료 처리

매 10분 : 배송 상태 동기화

매 1시간 : 인기 상품 랭킹 집계

매일 00시 : 쿠폰/포인트 만료 처리

매일 02시 : 매출 통계 집계

18. 기술 스택 제안

18.1 백엔드

- Java 21
- Spring Boot
- Spring Security
- Spring Data JPA
- QueryDSL
- Gradle

18.2 데이터 저장소

- MySQL 또는 PostgreSQL
- Redis
- Elasticsearch 또는 OpenSearch
- Object Storage

18.3 비동기 처리

- Kafka 또는 RabbitMQ
- 초기에는 Spring Event + 비동기 큐로 시작 가능
- 트래픽 증가 시 Kafka 도입

18.4 인프라

- Docker
 - Kubernetes 또는 ECS
 - Nginx 또는 API Gateway
 - CI/CD 파이프라인
 - Prometheus
 - Grafana
 - ELK Stack 또는 OpenSearch Dashboard
-

19. 초기 개발 우선순위

19.1 1단계: MVP

가장 먼저 구현해야 하는 기능이다.

- 회원가입 / 로그인
- 상품 등록 / 조회
- 상품 옵션 관리
- 재고 관리
- 장바구니
- 주문 생성
- 결제 연동
- 주문 조회
- 관리자 상품 관리

19.2 2단계: 운영 기능 강화

- 쿠폰
- 포인트
- 리뷰
- 배송사 연동
- 관리자 주문 관리
- 검색 엔진 도입
- 주문/결제 모니터링

19.3 3단계: 고도화

- 개인화 추천
 - 랭킹 시스템
 - 기획전
 - 라이브 특가
 - 매장 재고 연동
 - 대규모 이벤트 대응 구조
 - MSA 분리
-

20. 설계상 중요한 의사결정

20.1 초기에는 모듈형 모놀리식으로 시작한다

초기부터 MSA로 가면 서비스 간 통신, 배포, 모니터링, 장애 대응 복잡도가 커진다.

따라서 처음에는 하나의 애플리케이션 안에서 도메인 모듈을 명확히 나누고, 나중에 트래픽과 조직 규모가 커졌을 때 주문, 결제, 상품, 검색 도메인을 독립 서비스로 분리한다.

20.2 주문 시점의 상품 정보를 주문 상세에 복사한다

상품명, 옵션명, 가격은 시간이 지나면 변경될 수 있다.

하지만 주문 내역은 사용자가 구매한 당시의 정보가 보존되어야 한다.

따라서 주문 상세에는 상품 ID뿐 아니라 주문 당시의 상품명, 옵션명, 가격을 함께 저장한다.

20.3 재고는 옵션 단위로 관리한다

커머스에서 실제 판매 가능 여부는 상품 단위가 아니라 옵션 단위로 결정된다.

예를 들어 같은 립스틱 상품이라도 1호 색상은 품절이고 2호 색상은 판매중일 수 있다.

따라서 재고는 product가 아니라 product_option 기준으로 관리한다.

20.4 결제는 반드시 멍등하게 처리한다

결제 승인 요청이나 PG 콜백은 네트워크 문제로 중복 발생할 수 있다.

같은 결제가 두 번 승인되거나 같은 주문이 여러 번 결제 완료 처리되면 안 된다.

따라서 paymentKey, orderNo, idempotencyKey를 기준으로 중복 처리를 막아야 한다.

20.5 재고 선점 방식을 사용한다

결제 과정은 사용자가 외부 결제창을 거치기 때문에 시간이 걸린다.

이때 재고를 완전히 차감하면 결제 이탈 시 재고 복구가 자주 발생한다.

반대로 아무 처리도 하지 않으면 결제 완료 시점에 품절이 발생할 수 있다.

따라서 주문 생성 시 일정 시간 동안 재고를 선점하고, 결제 완료 시 확정 차감하는 방식이 적절하다.

21. 예상 확장 방향

21.1 MSA 분리 기준

트래픽이 증가하거나 배포 단위 분리가 필요해지면 다음 순서로 서비스를 분리할 수 있다.

1. Search Service
2. Product Service
3. Order Service
4. Payment Service
5. Inventory Service
6. Promotion Service
7. Delivery Service
8. Review Service

검색은 독립성이 높고 조회 트래픽이 많기 때문에 가장 먼저 분리하기 좋다.

주문, 결제, 재고는 정합성이 중요하므로 충분한 이벤트 설계와 트랜잭션 전략을 마련한 뒤 분리하는 것이 안전하다.

21.2 대규모 트래픽 대응

- 상품 목록 캐싱
- CDN을 통한 이미지 제공
- 검색 엔진 샤딩
- DB read replica 도입
- 주문 API rate limit
- 이벤트 큐 기반 비동기 처리
- 인기 상품 재고 처리 전용 큐 도입

22. 최종 요약

본 설계는 올리브영과 같은 헬스·뷰티 커머스 플랫폼을 만들기 위한 백엔드 구조를 제안한다.

핵심은 다음과 같다.

- 상품, 옵션, 재고를 명확히 분리한다.
- 주문 시점의 가격과 상품 정보를 주문 상세에 보존한다.
- 결제는 PG사 연동과 콜백을 기준으로 확정한다.
- 재고는 선점 후 결제 완료 시 확정 차감한다.
- 쿠폰과 포인트는 이력 기반으로 관리한다.
- 검색은 초기에는 DB로 시작하되, 확장 시 검색 엔진을 도입한다.
- 초기 구조는 모듈형 모놀리식으로 시작하고, 필요 시 MSA로 분리한다.
- 이벤트 기반 처리를 통해 주문 이후 부가 작업을 비동기화한다.
- 관리자 백오피스를 통해 상품, 주문, 재고, 프로모션을 운영 가능하게 한다.

이 구조는 초기 개발 속도와 향후 확장성을 모두 고려한 현실적인 커머스 백엔드 설계안이다.